Beatrice - Mini Portfolio

> **A privacy-first, end-to-end encrypted chat application that lives entirely in the terminal.** The server routes your messages but can *never* read them.

Stack: Go · HPKE / post-quantum ML-KEM · WebSockets · SQLite · Bubble Tea (TUI)

Project Overview

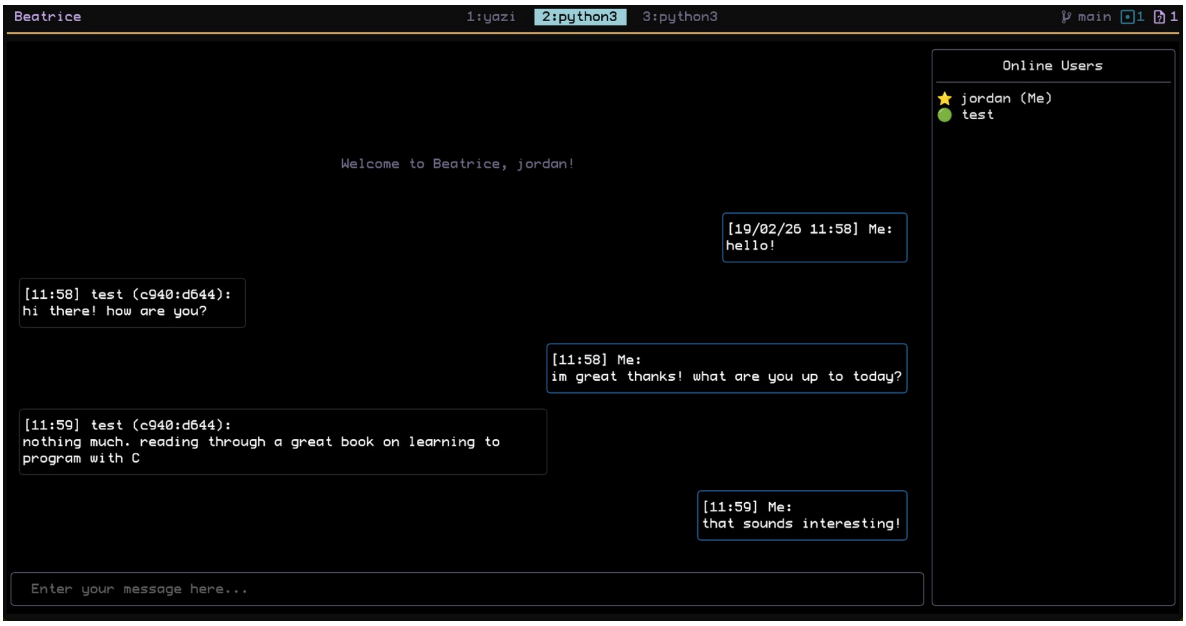
Beatrice is a real-time chat app for people who live in the terminal. Conversation happens in a TUI, and the server routing every message is cryptographically unable to decrypt a single one of them.

It began as a Python/Textual prototype built to learn **network security, async I/O, and cryptography**, and is now being rewritten in Go with a focus on performance, type safety, and modern (post-quantum) primitives.

What makes it interesting

Area	Why it matters
Post-quantum crypto	Uses **ML-KEM-768 + X25519**, HKDF-SHA512, AES-256-GCM. Ready for the quantum era, today.
Zero-trust server	The server routes ciphertext it cannot decrypt.
Custom wire protocol	A typed, versioned envelope for every packet travelling between client and server.
Resilient async client	Exponential backoff with jitter keeps the client alive through network hiccups.
Concurrent hub	A single event-loop goroutine handles all client lifecycle without locks.
Encrypted at rest	Full ciphertext + public key + IVs persisted to SQLite with proper FK constraints.

Screenshot 1 — The app in action (python version)



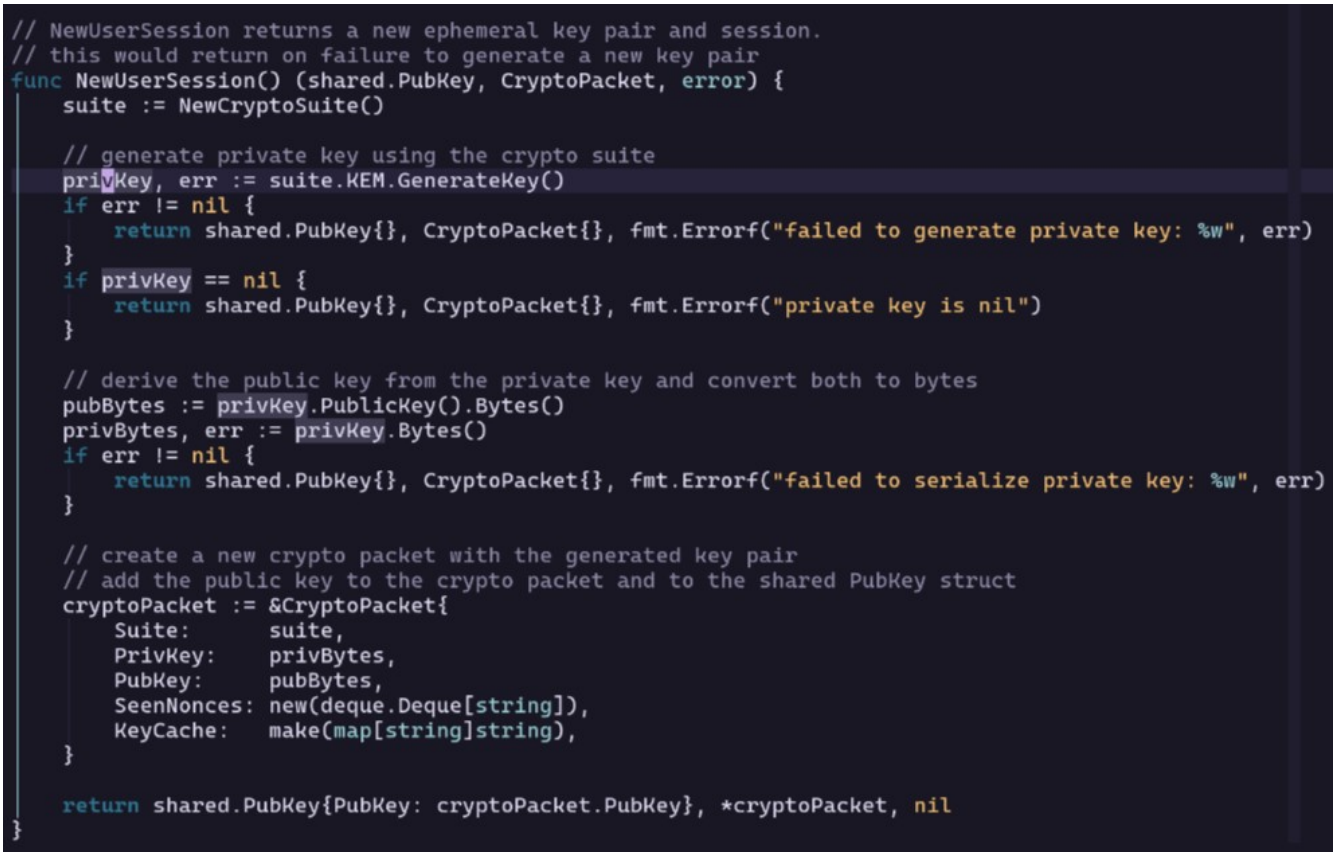
This shows the Python version of the app's launch page and the chat view. Yet to be implemented in the Go rewrite.

Screenshot 2 - The post-quantum session construction

Source: internal/client/crypto_utils.go

```
func NewCryptoSuite() SuiteConfig {
   return SuiteConfig{
      KEM:  hpke.MLKEM768X25519(),
      KDF:  hpke.HKDFSHA512(),
      AEAD: hpke.AES256GCM(),
      Info: []byte("Beatrice"),
   }
}
```

The heart of the project. Each session spins up an **ephemeral key pair**, derives its public half, and wires a configurable HPKE suite whose defaults target **post-quantum security** (ML-KEM-768 hybrid) - with the key exchange itself providing forward secrecy. The CryptoPacket also carries a rolling SeenNonces deque and a KeyCache, so every session is one-time and no nonce is ever reused.



```go
// NewUserSession returns a new ephemeral key pair and session.
// this would return on failure to generate a new key pair
func NewUserSession() (shared.PubKey, CryptoPacket, error) {
    suite := NewCryptoSuite()

    // generate private key using the crypto suite
    privKey, err := suite.KEM.GenerateKey()
    if err != nil {
        return shared.PubKey{}, CryptoPacket{}, fmt.Errorf("failed to generate private key: %w", err)
    }
    if privKey == nil {
        return shared.PubKey{}, CryptoPacket{}, fmt.Errorf("private key is nil")
    }

    // derive the public key from the private key and convert both to bytes
    pubBytes := privKey.PublicKey().Bytes()
    privBytes, err := privKey.Bytes()
    if err != nil {
        return shared.PubKey{}, CryptoPacket{}, fmt.Errorf("failed to serialize private key: %w", err)
    }

    // create a new crypto packet with the generated key pair
    // add the public key to the crypto packet and to the shared PubKey struct
    cryptoPacket := &CryptoPacket{
        Suite:      suite,
        PrivKey:    privBytes,
        PubKey:     pubBytes,
        SeenNonces: new(deque.Deque[string]),
        KeyCache:   make(map[string]string),
    }

    return shared.PubKey{PubKey: cryptoPacket.PubKey}, *cryptoPacket, nil
}
```

Screenshot 3 - The wire protocol

Source: internal/shared/types.go

```
type GeneralPacket struct {          // global envelope
    Type    string          `json:"t"`
    Message json.RawMessage `json:"m"`
}

type MessagePacket struct {
    Recipient string `json:"r"`
    Sender    string `json:"s"`
    IV        string `json:"iv"`  // Base64 IV for AES-GCM
    AESKey    string `json:"k"`   // Ephemeral AES key wrapped by recipient's pub key
    Content   string `json:"c"`   // Encrypted payload
    Signature string `json:"sig"` // Non-repudiation check
}
```

Every message is a thin, typed envelope whose inner payload is opaque JSON. The design keeps the protocol explicit and forward-compatible: the sender's signature proves authenticity, the per-message AES key is wrapped for the recipient only, and the payload travels over the socket without the server ever holding a plaintext copy.

Screenshot 4 - Resilience: exponential backoff with jitter

Source: internal/client/websocket.go

```
for {
    conn, _, err := websocket.Dial(ctx, addr, nil)
    if err != nil {
        jitter := time.Duration(rand.Int63n(int64(delay / 2)))
        wait := delay + jitter
        select {
        case <-time.After(wait):
        case <-ctx.Done():
            return User{}, ctx.Err()
        }
        delay = min(delay*2, maxDelay)
        continue
    }
    // ...
    client := User{Conn: conn, TuiChan: make(chan []byte, 100)}
    if err := client.readLoop(ctx); err != nil { /* ... */ }
    return client, nil
}
```

Reconnection logic that not only backs off exponentially (up to a cap) but jitters each retry so a burst of reconnecting clients naturally de-synchronises rather than causing a thundering herd on the server. Context-aware throughout, so cancellation

aborts a wait cleanly. The readLoop pushes frames into a buffered channel bound for the TUI - a clean, decoupled producer/consumer structure.

Screenshot 5 - Lock-free concurrency on the server hub

Source: internal/server/websocket.go

```
case client, ok := ←h.deleteClient:
   if !ok {
      return
   }
   delete(h.clients, client)
   client.CloseNow()
```

The server funnels every lifecycle event - register, broadcast, deregister - through Go channels into one listener goroutine. This is the idiomatic Go way to share state by communicating rather than locking; no mutexes, no race conditions, and each case is a linearised decision point. The consumer loop also carries a 30-second read timeout per wait, so stale connections are cleaned up automatically.

```go
// wsHandler handles incoming websocket connections
// it distributes the connection to the appropriate channels
func (h *Hub) wsHandler(w http.ResponseWriter, r *http.Request) {
    conn, err := websocket.Accept(w, r, nil)
    if err != nil {
        slog.Error("Error accepting websocket connection", "err", err)
        return
    }

    // start listener goroutine to handle incoming packets accordingly
    ctx := conn.CloseRead(r.Context())
    go h.Listener(ctx)

    // send conn to the hub for handshake etc
    h.register <- conn

    // defer closing the connection
    defer func() {
        h.deleteClient <- conn
    }()

    // blocks until a message is received
    for {
        ctx, cancel := context.WithTimeout(ctx, 30*time.Second)
        _, msg, err := conn.Read(ctx)
        cancel()
        if err != nil {
            slog.Error("Error reading from websocket connection", "err", err)
            break
        }

        // send msg to the hub
        h.broadcast <- msg
    }
}
```

Screenshot 6 - Encrypted-at-rest schema and sanitisation

Source: internal/server/db.go + validation.go

```
CREATE TABLE IF NOT EXISTS messages (
   id              INTEGER PRIMARY KEY AUTOINCREMENT,
   sender          TEXT NOT NULL,
   recipient       TEXT NOT NULL,
   iv              TEXT NOT NULL,
   encrypted_key     TEXT NOT NULL,
   encrypted_content  TEXT NOT NULL,
   signature         TEXT NOT NULL,
   timestamp         TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
   FOREIGN KEY (sender) REFERENCES users(nickname)
);
```

The schema stores nothing but ciphertext and its keying material - persistence is encrypted-at-rest by construction. Foreign keys are enabled at the DSN layer, and

a UsernameSanitizer strips non-alphanumerics before anything touches the database or wire, giving defence-in-depth across the whole attack surface.

Tech Stack

- **Go 1.26** – main rewrite language, static binary server + client
- **crypto/hpke** – ML-KEM-768 with X25519 · HKDF-SHA512 · AES-256-GCM
- **coder/websocket** – long-lived duplex transport
- **modernc.org/sqlite** – pure-Go, embedded persistence (no CGO)
- **charmbracelet/bubbletea + lipgloss** – the TUI
- **[Python origins]** – original asyncio + Textual + pycryptodome prototype, still in src/